



Tecnológico de Monterrey

TE3003B: Integración de Robótica y Sistemas Inteligentes

Campus Monterrey

Módulo 2 : Challenge - Primera entrega

Jesús Alberto García González | A01369587

Ruben de Jesus Hilario Cruz | A00835860

Uriel Ernesto Lemus Raygoza | A00835767

Felipe de Jesús García García | A01705893

Monterrey, Nuevo León, México, Abril 30, 2026

1. Investigación : control de motores en carga

Antes de proponer un enfoque para el control de nuestro montacargas, se revisaron tres patentes que abordan el problema desde distintos ángulos: detección de la posición vertical de las horquillas, control eléctrico del motor de elevación mediante PWM, y arquitectura distribuida del sistema de control.

1.1. Forklift safety sensor and control system (US 7,287,625 B1)

- a) **Problema a resolver:** En esta patente el problema principal es que las horquillas del montacargas sufren desgaste mecánico cuando se opera, por lo que hay una pérdida del 10% del material en la zona inferior, por lo que la capacidad de carga se reduce a un 20%.
- b) **Innovación principal:** Se propone un sistema que detecta cuando las horquillas entran en contacto con el suelo y procede a bloquear el mecanismo de tracción, evitando que el carro se eleve o sufra desgaste.
- c) **Componentes físicos:** Sensor de posición vertical(sensor de proximidad), un controlador, dispositivo de bloqueo y el mástil del montacargas.
- d) **Enfoque para identificar posiciones:** Los componentes principales son interruptores de proximidad montados en una posición fija del mástil. Cuando la horquilla pasa por una altura predefinida, el sensor cambia de estado y notifica al controlador y genera una interrupción.
- e) **Relevancia para el challenge:** La idea de mapear o identificar la posición del mástil en estados discretos(arriba,abajo, paro) coincide directamente con el enfoque del challenge, ya que usamos 2 bits de un DIP switch para seleccionar entre tres estados del motor. El concepto de un controlador que recibe señales y produce salidas de actuación es exactamente lo que buscamos implementar en la FPGA

1.2. Apparatus for controlling a pump motor of a forklift truck (US 6,041,163)

- a) **Problema a resolver:** En esta patente se enfoca principalmente en los montacargas eléctricos antiguos, ya que el motor de la bomba hidráulica se controlaba con encendido/apagado simple, lo que generaba ruido irregular bajo carga ligera y un consumo poco eficiente cuando la carga variaba.
- b) **Innovación principal:** La propuesta es un controlador que regula el torque del motor en tiempo real mediante una señal PWM cuyo ciclo de trabajo varía en función de la corriente medida en el motor. Esto reduce el ruido y adapta la respuesta al peso real de la carga.
- c) **Componentes físicos:** Batería, driver que actúa como puente, motor de bomba DC, sensor de corriente, y controlador electrónico que ejecuta el lazo cerrado.
- d) **Enfoque para identificar posiciones:** En lugar de detectar posiciones discretas, este sistema mide carga mediante la corriente del motor y modula el ancho de pulso de la señal PWM. Cuando la carga aumenta, el duty aumenta, y viceversa, evitando ruido innecesario.
- e) **Relevancia para el challenge:** Esta patente nos ayudó a entender cómo nos ayuda y funciona el control PWM por puente H para motores DC. Pues la

técnica de generar PWM con periodo fijo y duty alto reproduce a escala didáctica este mismo principio.

1.3. Distributed control system for forklift (US 7,366,600)

- a) **Problema a resolver:** En montacargas modernos se busca integrar subsistemas heterogéneos, donde centralizar el control en un solo microcontrolador genera cableado masivo, baja tolerancia a fallas.
- b) **Innovación principal:** Se propone una arquitectura distribuida donde varios controladores se comunican por una red interna. Cada nodo procesa localmente las señales de los sensores que le competen y publica datos al resto del sistema.
- c) **Componentes físicos:** Controlador del motor, controlador del vehículo, unidad de display, sensores de pedal, motor y el bus de red.
- d) **Enfoque para identificar posiciones:** Las posiciones y mecanismos se traducen en mensajes digitales. El controlador del motor ajusta el voltaje aplicado al motor en respuesta a la combinación de señales recibidas.
- e) **Relevancia para el challenge:** Aunque el challenge usa solo la FPGA como nodo principal, si también se añade el uso de la Jetson se entiende mejor la idea por capas, donde la Jetson se encarga de lo más “inteligente” (decisiones o visión), la FPGA del control lógico, el L298N de la potencia y el motor DC de la acción. Básicamente es una versión simple de un sistema distribuido bien organizado.

2. Especificación mecánica del forklift

El montacargas incorporado al Puzzlebot es un modelo a escala diseñado para elevar y descender una pequeña plataforma a lo largo de un riel vertical ubicado en la parte frontal del robot. El sistema convierte el movimiento rotacional de un motor DC en un desplazamiento lineal mediante un mecanismo de hilo y polea. El hilo de nylon se enrolla en el eje del motor, asciende por el mástil, pasa por una polea en la parte superior y desciende hasta el carro móvil. De esta forma, cada giro del motor, controlado mediante PWM, se traduce directamente en el movimiento vertical de la plataforma, permitiendo levantar o bajar cargas de manera controlada y estable.

Componente	Dimensiones / Especificación	Descripción
Mástil	~ 200 x 80 x 50 mm, plástico/MDF	Estructura vertical que guía el movimiento lineal de nuestro puzzle bot
Puzzlebot	~12 × 11 × 10 cm, acrílico	Parte móvil que transporta la carga
Motor DC	Motor 5 V, con encoder	Genera movimiento rotacional
Hilo de nylon + Polea superior	Polea de ~ 12 mm de diámetro,	Transmite la rotación a movimiento lineal

	hilo de nylon de 0.5 mm	
--	-------------------------	--

Especificación mecánica del forklift

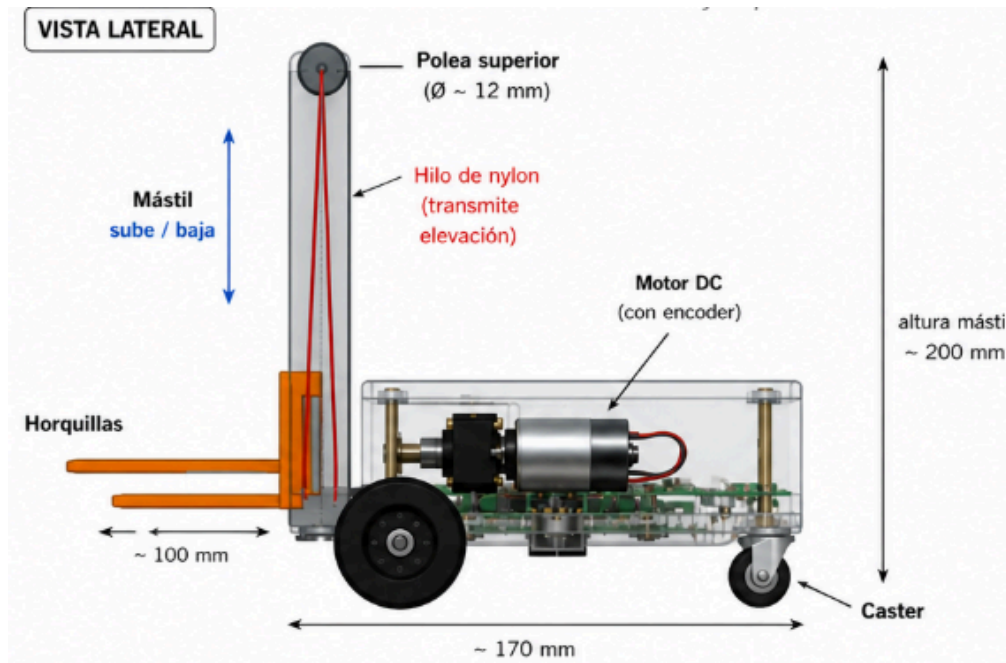


Figura 1. Vista lateral del forklift con dimensiones aproximadas y componentes principales

Integración entre partes :

- **Motor DC - cable :** El motor DC enrolla y desenrolla el hilo de nylon, convirtiendo el movimiento rotacional en tracción lineal para elevar o descender el sistema.
- **Polea - horquillas :** La polea redirige el hilo de nylon, permitiendo que el movimiento se transmita verticalmente hacia las horquillas de manera suave y controlada.
- **Puzzlebot - Mástil:** El mástil está fijado al chasis del Puzzlebot, proporcionando una guía rígida y estable para el movimiento vertical del sistema de elevación.
- **FPGA - Puente H - Motor :** La FPGA genera la señal PWM y la dirección de giro, el puente H adapta esta señal para controlar la potencia entregada al motor DC, permitiendo regular velocidad y sentido de movimiento.

3. Enfoque propuesto para el control del forklift

3.1. Propósito del sistema:

Nuestro sistema controla el sentido de giro de un motor DC de 5V (que mueve el montacargas) usando como controlador una FPGA Tang Nano 20K y como etapa de potencia el módulo L298N. El usuario selecciona el estado deseado (parar, subir, bajar) por medio de dos bits de un DIP switch, y la FPGA traduce esa entrada a tres señales digitales (IN1, IN2 y ENA) que comandan al L298N. La señal ENA es una PWM de 50 Hz con duty al 90 % (pulso de 18 ms en periodo de 20 ms).

Algo muy importante que nos dimos cuenta es que en el enunciado original se proponen tres anchos de pulso (1.0, 1.5 y 2.0 ms sobre periodo de 20 ms) que son los típicos de un servomotor. Sin embargo, esos duty cycles son demasiado bajos para arrancar un motor DC con reductor. Por ese motivo, en este sistema el duty del PWM se eleva al 90 % cuando el motor está activo.

3.2. Entradas y salidas:

Tipo	Señal	Detalles
Entrada	PB0	Bit 0 del DIP switch, LVCMOS 3.3 V
Entrada	PB1	Bit 1 del DIP switch, LVCMOS 3.3 V
Entrada	clk_27mhz	Reloj on-board del Tang Nano 20K
Salida	I298_in1	Dirección 1 del L298N (LVCMOS 3.3 V)
Salida	I298_in2	Dirección 2 del L298N (LVCMOS 3.3 V)
Salida	I298_ena	PWM de habilitación del L298N (LVCMOS 3.3 V, 50 Hz, 90 % duty)

3.3. Tabla de estados:

PB1	PB0	Estado	IN1	IN2	ENA
0	0	Paro	0	0	Bajo (sin PWM)
0	1	Arriba	1	0	PWM 90 % duty
1	0	Abajo	0	1	PWM 90 % duty
1	1	Paro	1	1	Bajo (sin PWM)

3.4. Requisitos funcionales:

- El sistema debe generar una señal PWM con periodo de 20 ms (50 Hz) en el pin ENA cuando el motor esté activo.
- Cuando PB1=0 y PB0=0, las tres salidas deben de estar en un nivel bajo (motor detenido).
- Cuando PB1=0 y PB0=1, IN1 debe estar en alto, IN2 en bajo, y ENA con PWM al 90% (motor gira a un lado).
- Cuando PB1=1 y PB0=0, IN1 debe estar en bajo, IN2 en alto, y ENA con PWM al 90% (motor gira al otro lado).
- Cuando PB1=1 y PB0=1, ENA debe estar en bajo (motor detenido por brake).

3.5. Requisitos no funcionales:

- Las entradas y las tres salidas del FPGA usan estándar LVCMOS33 (3.3 V).
- El motor y el L298N se alimentan desde una fuente externa de 7-12 V, separada de la Tang Nano.
- Debe existir un GND común entre la FPGA, el L298N y la fuente externa.
- El diseño debe ser sintetizable en Gowin EDA para el chip GW2AR-LV18 del Tang Nano 20K.

3.6. Diagrama UML de despliegue:

En el siguiente diagrama UML se describen los nodos físicos del sistema, lo que compone a cada uno de estos, y los protocolos de las interfaces entre ellos.

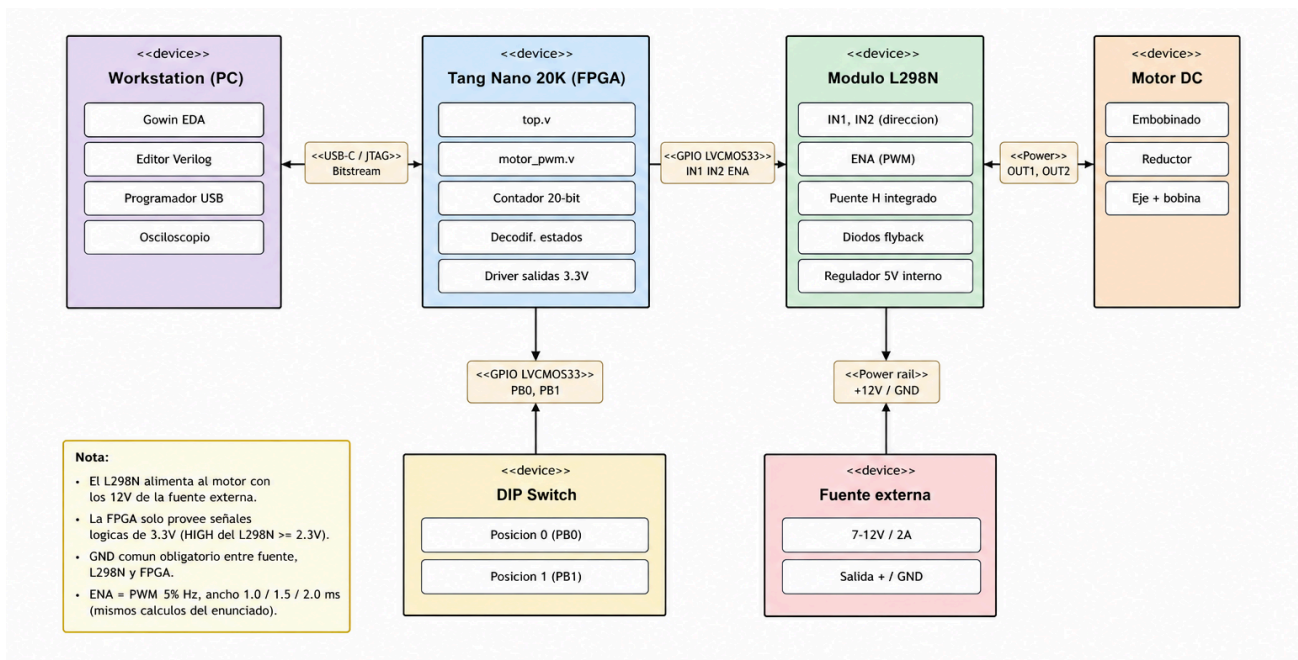


Figura 2. Diagrama UML de despliegue del sistema de control del forklift

3.7. Justificación de los componentes:

- **FPGA Tang Nano 20k:** Es ideal porque permite trabajar en paralelo sin depender de software. Su reloj de 27 MHz da muy buena resolución de tiempo y tiene suficientes pines GPIO para manejar todo sin problema.
- **DIP switch(2 posiciones):** Es una forma simple y confiable de seleccionar estados. No hay problema de rebote en uso normal y permite definir fácilmente las combinaciones para controlar el motor.
- **Modulo L298:** Se usa como puente H para manejar el motor. Ya incluye protecciones como diodos flyback y facilita mucho la conexión del motor.
- **Motor DC:** Da el torque necesario para levantar el sistema a baja velocidad. Es más sencillo que usar un servo porque no necesitas control de posición, solo controlar el voltaje para moverlo.
- **Fuente externa:** El motor consume más corriente de la que puede dar la FPGA por USB, así que se necesita una fuente aparte. Esto evita caídas de voltaje y que la FPGA se reinicie.

4. Conexiones de cableado y archivo de constraints

El siguiente diagrama muestra la interconexión general del sistema de control, diferenciando claramente entre la etapa de potencia y la de control. La alimentación de potencia se suministra mediante una fuente externa, mientras que las señales de control se generan desde la unidad principal y se envían hacia la etapa de actuación.

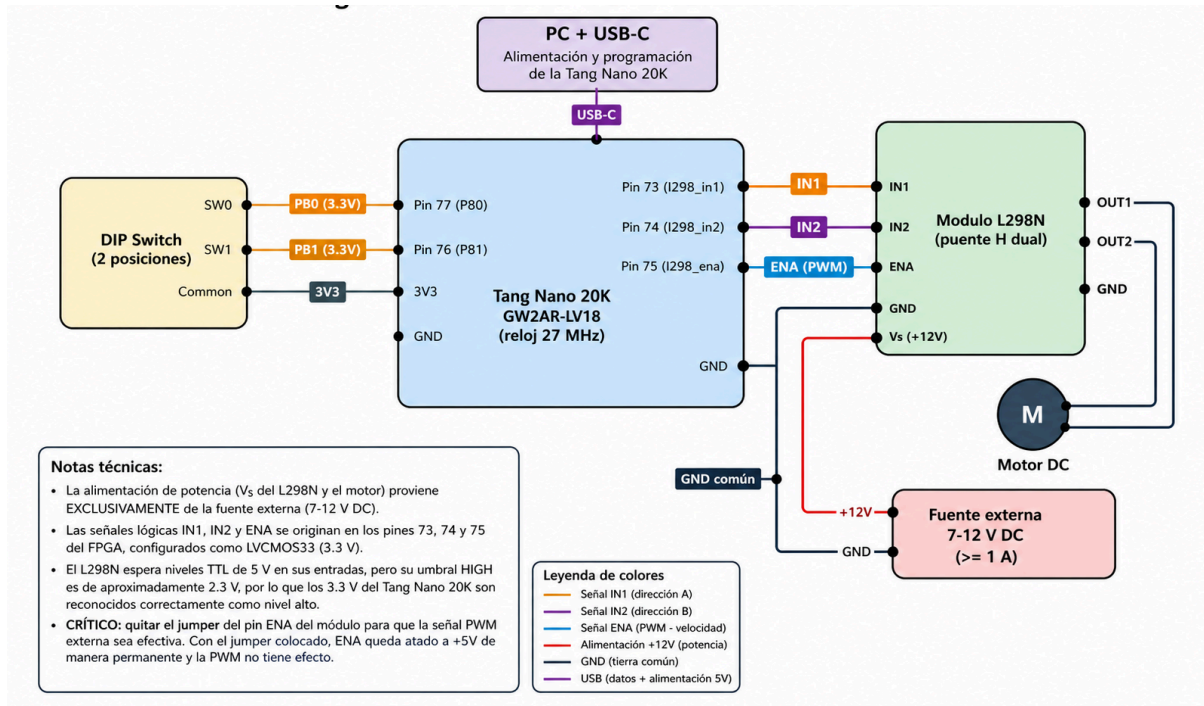


Figura 3. Diagrama de cableado completo del sistema

4.1. Asignación física de pines

Señal	Pin Tang Nano	Dirección	Estándar	Pull/Drive
clk_27mhz	4	Entrada	LVCMOS33	PULL_UP(oscilador)
pb0	77	Entrada	LVCMOS33	PULL_DOWN
pb1	76	Entrada	LVCMOS33	PULL_DOWN
I298_in1	73	Salida	LVCMOS33	DRIVE = 8mA
I298_in2	74	Salida	LVCMOS33	DRIVE = 8mA
I298_ena	75	Salida	LVCMOS33	DRIVE = 8mA

4.2. Listado del archivo de constraints

A continuación se muestra el contenido del archivo de restricciones físicas utilizado en el diseño. No se incluye una señal de reset, ya que el sistema no la requiere: los registros se inicializan automáticamente al cargar la configuración. Esto simplifica la implementación al no necesitar cableado adicional y evita posibles problemas por ruido eléctrico, especialmente en entornos donde hay motores involucrados.

```
//=====
// File:      tangnano20k.cst
// Project:    TE3003B - Forklift Motor Control Challenge (Delivery 1)
// Target:     Tang Nano 20K (GW2AR-LV18QN88PC8/I7)
// Description: Physical Constraints File - pin assignments for Gowin EDA.
//             Three outputs go to the L298N H-bridge module: IN1, IN2 and
//             ENA. The L298N typically expects 5V TTL logic at its inputs,
//             but its HIGH threshold is around 2.3V, so the 3.3V LVCMOS33
//             outputs of the Tang Nano 20K drive it reliably.
//=====
// ----- Clock -----
// On-board 27 MHz oscillator
IO_LOC "clk_27mhz" 4;
IO_PORT "clk_27mhz" PULL_MODE=UP IO_TYPE=LVCMOS33;

// ----- Reset -----
// On-board user button S1
IO_LOC "rst_n" 88;
IO_PORT "rst_n" PULL_MODE=UP IO_TYPE=LVCMOS33;

// ----- DIP switch inputs -----
// PB0 and PB1 are wired to two of the DIP switch positions.
IO_LOC "pb0" 77;
IO_PORT "pb0" PULL_MODE=DOWN IO_TYPE=LVCMOS33;

IO_LOC "pb1" 76;
IO_PORT "pb1" PULL_MODE=DOWN IO_TYPE=LVCMOS33;

// ----- L298N H-bridge control outputs -----
// IN1 / IN2 set the rotation direction of the motor.
// ENA carries the PWM signal that controls the motor speed.
IO_LOC "l298_in1" 73;
IO_PORT "l298_in1" IO_TYPE=LVCMOS33 DRIVE=8;

IO_LOC "l298_in2" 74;
IO_PORT "l298_in2" IO_TYPE=LVCMOS33 DRIVE=8;

IO_LOC "l298_ena" 75;
IO_PORT "l298_ena" IO_TYPE=LVCMOS33 DRIVE=8;
```

5. Cálculo de cuentas para la señal PWM

La Tang Nano 20K dispone de un oscilador interno de 27 MHz que se utiliza como reloj principal del módulo. El periodo de cada cuenta es por tanto:

$$T_{clk} = 1 / 27,000,000 \text{ Hz} = 37.0370 \text{ ns por cuenta}$$

5.1. Procedimiento para el periodo 20 ms:

Se requiere generar una señal PWM con periodo de 20 ms (frecuencia de 50 Hz). El número de cuentas necesario es:

$$N_{\text{periodo}} = 20 \text{ ms} / 30.0370 \text{ ns} = 540,000 \text{ cuentas}$$

$$\text{Equivalente: } 0.020 \text{ s} \times 10^6 \text{ Hz} = 540,000 \text{ cuentas}$$

5.2. Cálculos pedidos y ancho de pulso utilizado

Para fines de documentación y comparación, decidimos mantener los cálculos para un servomotor, pero en nuestro caso nosotros usamos motor DC, por lo que el resultado es diferente para ambos casos:

Pulso de referencia	Cálculo	Cuentas	Duty cycle
1.0 ms (servo: -90 grados)	0.001 s x 27 MHz	27, 000	5 %
1.5 ms (servo: 0 grados)	0.0015 s x 27 MHz	40, 500	7.5 %
2.0 ms (servo: +90 grados)	0.002 s x 27 MHz	54, 000	10 %

Sin embargo, en pruebas confirmamos que con esos duty cycles son bajos para que el motor DC logre arrancar, ya que la energía promedio entregada por la PWM no vence la fricción estática del tren de engranajes. Por eso, en el módulo final se utiliza un ancho de pulso del 90 % (18 ms), calculado como:

Pulso ON	Cálculo	Cuentas	Duty cycle
18 ms	0.018 s x 27 MHz	486,000	90%

5.3. Tamaño del contador:

El valor máximo a contar es 540,000. Como $2^{19} = 524,288 < 540,000$, se requiere un contador de 20 bits. En el código Verilog se declaró como reg [19:0] counter; esto reserva exactamente 20 flip-flops para el registro de cuenta.

5.4. Tabla de verdad final del sistema

PB1	PB0	Estado	IN1	IN2	ENA	CUENTAS(ENA)
0	0	Detenido	0	0	Bajo continuo	0
0	1	Arriba	1	0	PWM 90 %	486,000 de 540,000
1	0	Abajo	0	1	PWM 90 %	486,000 de 540,000
1	1	Detenido	1	1	Bajo continuo	0

6. Código Verilog y resultados

- 6.1. Listado de motor_pwm.v** : Este módulo es el encargado de generar las señales de control. La dirección se define de forma combinacional a partir de las entradas, lo que permite una respuesta inmediata sin retardos. La señal de habilitación utiliza un contador de 20 bits para fijar un periodo de 20 ms, junto con un comparador que genera una PWM con un duty cycle del 90%. Cuando ambas entradas están en el mismo estado, la señal de habilitación se desactiva, deteniendo el sistema.

```
module motor_pwm (
    input wire clk,
    input wire pb0,
    input wire pb1,
    output wire in1,
    output wire in2,
    output reg ena
);

// -----
// Direccion: el DIP switch va directo al puente H. PB1=PB0 mantiene
// el motor parado (ambos a 0 = coast; ambos a 1 = brake).
// -----
assign in1 = pb0;
assign in2 = pb1;

// -----
// PWM @ 50 Hz - periodo de 20 ms, duty 90%.
// Periodo: 20 ms -> 540,000 cuentas (a 27 MHz).
// Pulso ON: 18 ms -> 486,000 cuentas (90 % de duty).
// Se eligio 90 % para vencer la friccion del reductor; con duty
// menor el motor zumba pero no llega a girar.
// -----
localparam [19:0] PERIOD    = 20'd540_000;
localparam [19:0] PULSE_HIGH = 20'd486_000;

reg [19:0] counter = 20'd0;

always @(posedge clk) begin
    if (counter >= PERIOD - 1)
        counter <= 20'd0;
    else
        counter <= counter + 1'b1;
end

// motor_active = 1 cuando uno y solo uno de los DIP esta en alto.
// Asi se cancela el PWM cuando el motor esta detenido.
wire motor_active = pb0 ^ pb1;

always @(posedge clk) begin
    ena <= motor_active && (counter < PULSE_HIGH);
end
```

```
endmodule
```

6.2. Listado de top.v: Wrapper top-level que conecta los puertos físicos de la FPGA al módulo de control.

```
module top (  
    input wire clk_27mhz,  
    input wire rst_n,      // se queda como puerto pero no se usa internamente  
    input wire pb0,  
    input wire pb1,  
    output wire l298_in1,  
    output wire l298_in2,  
    output wire l298_ena  
);  
    motor_pwm u_motor_pwm (  
        .clk (clk_27mhz),  
        .pb0 (pb0),  
        .pb1 (pb1),  
        .in1 (l298_in1),  
        .in2 (l298_in2),  
        .ena (l298_ena)  
    );  
endmodule
```

6.3. Verificación en simulación: Antes de descargar el bitstream a la Tang Nano, verificamos el módulo en simulación usando un testbench que recorre las cuatro combinaciones del DIP switch. Las siguientes figuras muestran las tres señales (IN1, IN2 y ENA) que la FPGA envía al módulo L298N en cada estado, tal como se mediría con un osciloscopio de tres canales.

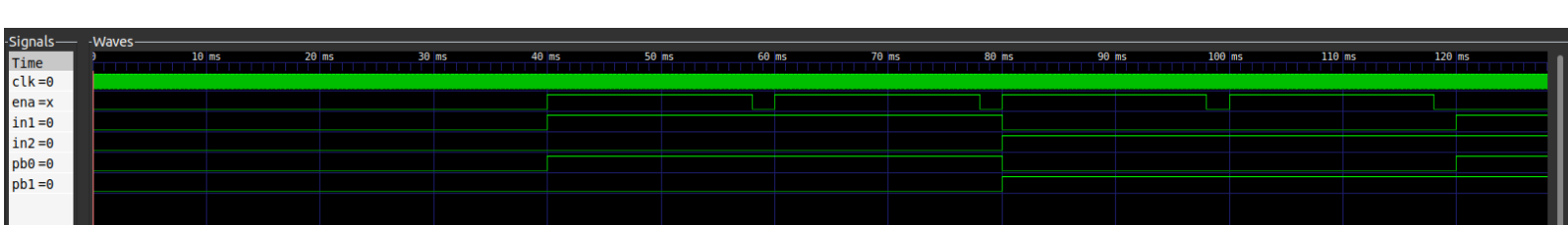


Figura 4. Todos las representaciones de los estados

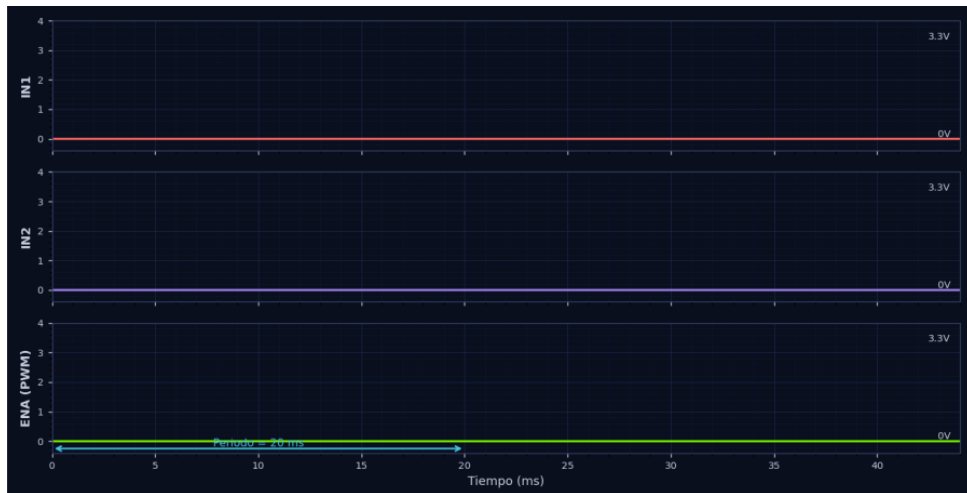


Figura 5. Estado DETENIDO (PB1=0, PB0=0): IN1=IN2=0, ENA en bajo continuo.

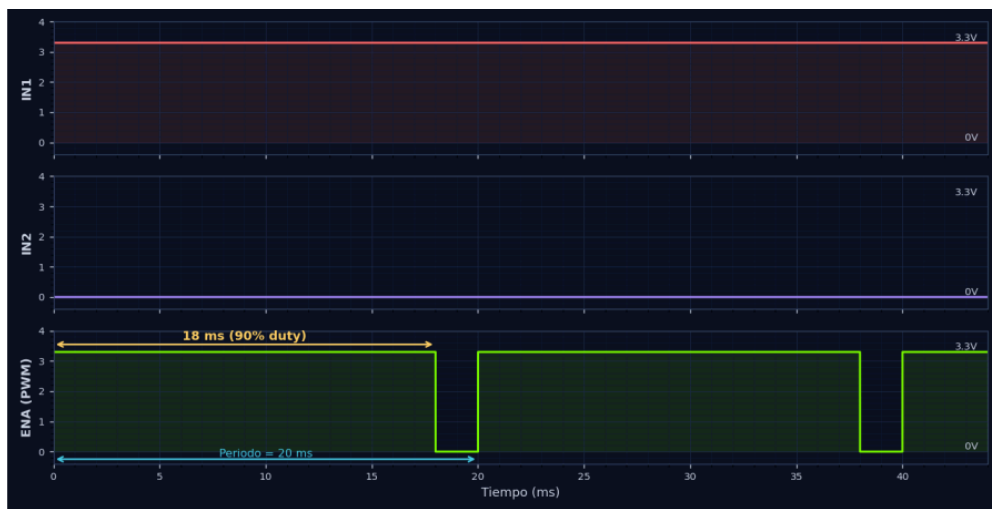


Figura 6. Estado SUBIR (PB1=0, PB0=1): IN1=1, IN2=0, ENA con PWM al 90 % de duty

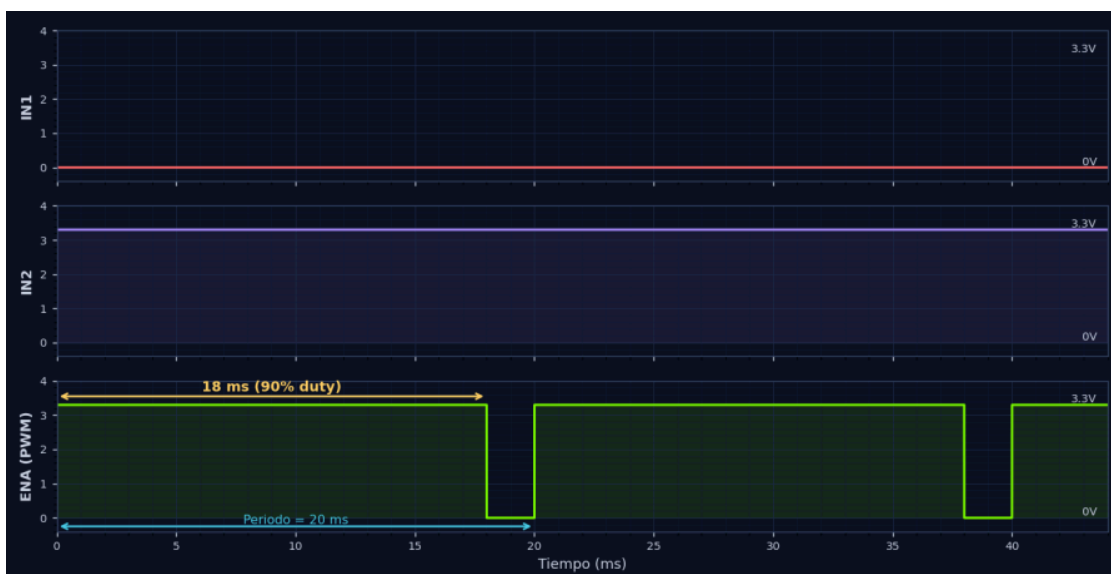


Figura 7. Estado BAJAR (PB1=1, PB0=0): IN1=0, IN2=1, ENA con PWM al 90 % de duty

7. Problemas encontrados y soluciones

Problema	Causa	Solución
El motor no se movía ni una sola vez con dutys cycle	La energía promedio entregada por una PWM con 5-10 % de duty es insuficiente para vencer la fricción estática del reductor del motor DC.	Aumentar el duty del PWM a 90 % (pulso de 18 ms en periodo de 20 ms). Ya que varía de un servo a un motor DC.
Las salidas no responden al cambio del DIP switch	Las entradas estaban flotantes (sin pull-down), generando ruido e interpretaciones incorrectas	Configurar PULL_MODE=DOWN en el archivo .cst para estabilizar las entradas
Fallas en el funcionamiento general	Conexiones incorrectas en el hardware y falta de tierra común	Corregir el cableado y asegurar un GND compartido entre todos los componentes

7.2. Observaciones experimentales

Durante las pruebas intentamos variar la velocidad del motor usando dos duty cycles distintos (95 % hacia adelante y 80 % hacia atrás). Aunque en el osciloscopio sí se veía claramente la diferencia en la PWM, en la práctica el motor giraba casi igual en ambos casos. Esto pasa principalmente por tres cosas: el reductor hace que pequeños cambios de velocidad no se noten, el L298N ya está cerca de su límite con duty altos, y además el sistema no tiene carga, entonces el motor alcanza su velocidad máxima muy rápido. Por eso decidimos dejar un duty fijo de 90 %, ya que funciona de forma estable y suficiente para el sistema. Más adelante, cuando haya carga real, el cambio de velocidad sí se va a notar mejor.

8. Conclusiones

En esta primera entrega se logró diseñar, sintetizar y probar un módulo en la FPGA Tang Nano para controlar un motor DC en tres estados (detenido, arriba, abajo), usando un DIP switch como entrada y el L298N como etapa de potencia. El sistema funciona de forma estable en hardware y el reloj de 27 MHz fue suficiente para generar la señal requerida. Uno de los principales aprendizajes fue entender la diferencia entre la teoría y la práctica. Por ejemplo, usar duty cycles bajos (como en servos) no funciona bien en motores DC con reductor, ya que no tienen suficiente energía para arrancar. Por eso se tuvo que aumentar el duty a valores altos (~90 %). También se observó que cambiar el duty entre 80 % y 95 % casi no afecta la velocidad visible del motor. Esto se debe al reductor, al comportamiento del

L298N y a que no hay carga en el sistema. En la práctica, el motor ya está cerca de su velocidad máxima.

Esto deja claro que no es lo mismo controlar un servo (posición) que un motor DC (velocidad/torque), y que el comportamiento real del sistema influye directamente en cómo se deben elegir los parámetros. A nivel general, el uso de PWM con puente H coincide con lo que se utiliza en aplicaciones reales, aunque aquí se implementa a pequeña escala.

9. Bibliografías

[1] B. Hudson, "Forklift safety sensor and control system," U.S. Patent 7,287,625 B1, oct. 30, 2007. Disponible en: <https://eureka.patsnap.com/patent-US7287625B1>

[2] H. S. Lee, "Apparatus for controlling a pump motor of a forklift truck," U.S. Patent 6,041,163, Mar. 21, 2000. [Online]. Available: <https://patents.google.com/patent/US604116>

[3] T. Ono y K. Otsuka, "Forklift and method for controlling induction motor applied to the same," U.S. Patent 7,235,948, jun. 26, 2007.

[4] STMicroelectronics, "L298 Dual Full-Bridge Driver Datasheet," documento DM00000240, 2000.

[5] Sipeed, "Tang Nano 20K Datasheet and Pinout," 2024. Disponible en: <https://wiki.sipeed.com/hardware/en/tang/tang-nano-20k/nano-20k.html>

[6] Gowin Semiconductor, "Gowin FPGA Products Data Sheet (GW2A series)," 2024.